Bounded Buffer

```
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>

struct Node {
   int data;
   struct Node* next;
};

struct Node *front = NULL, *rear = NULL;

sem_t empty, full, mutex;
int MAX = 5;
int item = 0;

void enqueue(int value) {
   struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
   temp->data = value;
   temp->next = NULL;

   if (rear == NULL) {
      front = rear = temp;
   } else {
      rear->next = temp;
      rear = temp;
   }
}
int dequeue() {
   if (front == NULL)
      return -1;
```

```
    struct Node* temp = front;
    int value = temp->data;

    front = front->next;
    if (front == NULL)
        rear = NULL;

    free(temp);
    return value;
}

int main() {
    int choice, value;

    sem_init(&empty, 0, MAX);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    while (1) {
        printf("\n1. Producer\n2. Consumer\n3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {

        case 1:
            if (sem_trywait(&empty) == 0) {
                sem_wait(&mutex);

                item++;
```

```
            enqueue(item);
            printf("Producer has produced item %d\n", item);

            sem_post(&mutex);
            sem_post(&full);
        } else {
            printf("Queue is FULL!\n");
        }
        break;

    case 2:
        if (sem_trywait(&full) == 0) {
            sem_wait(&mutex);

            value = dequeue();
            printf("Consumer has consumed item %d\n", value);

            sem_post(&mutex);
            sem_post(&empty);
        } else {
            printf("Queue is EMPTY!\n");
        }
        break;

    case 3:
        exit(0);

    default:
        printf("Invalid choice!\n");
    }
}
```

```
    return 0;
}
```

```
1. Producer
2. Consumer
3. Exit
Enter your choice: 1
Producer has produced item 1

1. Producer
2. Consumer
3. Exit
Enter your choice: 1
Producer has produced item 2

1. Producer
2. Consumer
3. Exit
Enter your choice: 2
Consumer has consumed item 1

1. Producer
2. Consumer
3. Exit
Enter your choice: 1
Producer has produced item 3

1. Producer
2. Consumer
3. Exit
Enter your choice: 3


=== Code Execution Successful ===
```

Bounded Buffer(Semaphore)

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
#include<semaphore.h>

#define BUFFER_SIZE 5
#define MAX_ITEMS 10   // limit output

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

int produced_count = 0;
int consumed_count = 0;

void* producer(void* arg) {
   int item;

   while (produced_count < MAX_ITEMS) {
      item = produced_count;

      sem_wait(&empty);
      pthread_mutex_lock(&mutex);

      buffer[in] = item;
      printf("Produced: %d at buffer[%d]\n", item, in);

      in = (in + 1) % BUFFER_SIZE;
```

```
        produced_count++;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        sleep(1);
    }

    return NULL;
}

void* consumer(void* arg) {
    int item;

    while (consumed_count < MAX_ITEMS) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumed: %d from buffer[%d]\n", item, out);

        out = (out + 1) % BUFFER_SIZE;
        consumed_count++;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        sleep(1);
    }

    return NULL;
```

```
}

int main() {
    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

```
Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Consumed: 1 from buffer[1]
Produced: 2 at buffer[2]
Consumed: 2 from buffer[2]
Produced: 3 at buffer[3]
Consumed: 3 from buffer[3]
Produced: 4 at buffer[4]
Consumed: 4 from buffer[4]
Produced: 5 at buffer[0]
Consumed: 5 from buffer[0]
Produced: 6 at buffer[1]
Consumed: 6 from buffer[1]
Produced: 7 at buffer[2]
Consumed: 7 from buffer[2]
Produced: 8 at buffer[3]
Consumed: 8 from buffer[3]
Produced: 9 at buffer[4]
Consumed: 9 from buffer[4]


=== Code Execution Successful ===
```

Dining Philosopher

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>

#define N 5

pthread_mutex_t forks[N];
pthread_t philosophers[N];

void* philosopher(void* num) {
   int id = *(int*)num;
   int left = id;
   int right = (id + 1) % N;

   while (1) {
      printf("Philosopher %d is thinking.\n", id);
      sleep(1);

      // Deadlock avoidance strategy
      if (id % 2 == 0) {
         pthread_mutex_lock(&forks[left]);
         printf("Philosopher %d picked up left fork %d.\n", id, left);

         pthread_mutex_lock(&forks[right]);
         printf("Philosopher %d picked up right fork %d.\n", id, right);
      } else {
         pthread_mutex_lock(&forks[right]);
         printf("Philosopher %d picked up right fork %d.\n", id, right);

         pthread_mutex_lock(&forks[left]);
```

```
            printf("Philosopher %d picked up left fork %d.\n", id, left);
        }

        printf("Philosopher %d is eating.\n", id);
        sleep(2);

        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);

        printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
    }

    return NULL;
}

int main() {
    int i;
    int ids[N];

    // Initialize forks
    for (i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }

    // Create philosopher threads
    for (i = 0; i < N; i++) {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    // Join threads
```

```
    for (i = 0; i < N; i++) {
        pthread_join(philosophers[i], NULL);
    }

    // Destroy mutexes (not reached)
    for (i = 0; i < N; i++) {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}
```

```
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 3 is thinking.
Philosopher 2 is thinking.
Philosopher 4 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 1 picked up right fork 2.
Philosopher 4 picked up left fork 4.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 0 picked up left fork 0.
Philosopher 3 picked up right fork 4.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 4 put down forks 4 and 0.
```